

Task Design for Crowdsourcing Complex Cognitive Skills

Gaoping Huang*
huang679@purdue.edu
Purdue University

West Lafayette, Indiana, United States

Meng-Han Wu*
wu784@purdue.edu
Purdue University

West Lafayette, Indiana, United States

Alexander J. Quinn
aq@purdue.edu
Purdue University

West Lafayette, Indiana, United States

ABSTRACT

Task design for crowdsourcing is a key factor limiting the quality of crowd-sourced results. This case study presents our design process for a complex cognitive task: generating Dimension/Values for categorizing ideas. Conveying the task to workers was a formidable design challenge. We present five strategies and lessons learned from testing with workers. Decomposing the cognitive process and testing mastery of each cognitive subprocess enabled us to finally convey the task requirements and obtain useful results.

CCS CONCEPTS

• **Human-centered computing** → **User interface design**; *Web-based interaction*.

KEYWORDS

crowdsourcing, human computation, instructions design

ACM Reference Format:

Gaoping Huang, Meng-Han Wu, and Alexander J. Quinn. 2021. Task Design for Crowdsourcing Complex Cognitive Skills. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3411763.3443447>

1 INTRODUCTION

Crowdsourcing has been proven successful for relatively simple cognitive tasks [7, 8]. However, there is a trend to ask crowd workers to do tasks that require more complex cognitive skills (e.g., [6]). BlueSky [5], for example, is a system that we developed to coordinate crowd workers to enumerate a uniform sample of ideas spanning an idea space. Specifically, the BlueSky system introduces a three-step workflow to guide workers to generate *Dimension/Values* (D/V) that lay the foundation for detecting redundant ideas and exploring idea space more comprehensively. The definition of D/V is given in the next section. While we are implementing the workflow, we keep receiving low-quality D/V from crowd workers. This is because generating valid D/V is an uncommon crowdsourcing task that requires relatively complex cognitive skills.

There are several potential factors that could affect the quality of results (e.g., incentives [9], workers' capability [4]), but we focus

on improving the task design to achieve better D/V quality. In particular, we tried a variety of strategies to alter the user interface and task instructions. In this process, we leveraged the lessons from the prior flawed iterations and eventually came up with a plausible solution.

In retrospect, we believe there were three aspects that are interesting and worth sharing with the community to inspire future instruction design for crowdsourcing.

Problem. The problem was interesting because we were asking participants to do a relatively complex cognitive skill for which there were no readily available examples to refer to, or words that would clearly describe what we wanted them to do. There were two main challenges. One challenge was with the vocabulary. We did not know which terms—attributes, categories, characteristics, dimensions, etc.—were more understandable by the workers because all of those terms are often overloaded, and sometimes have different and conflicting meanings depending on the context. Another challenge we had was that the task initially seems similar to other tasks that would be easier to think of (e.g., Cascade [2]). Workers are already accustomed to classifying items into a discrete set of categories, but the notion of choosing valid D/V with which *another worker* could categorize any future ideas is relatively uncommon. The task is creative and plausible to perform, but it is difficult to convey, especially without the benefit of back and forth communication to rectify any misunderstandings.

Process. The process was interesting because we tried a wide variety of approaches with no significant difference other than the way we were conveying what we wanted them to do. We tried a variety of input validation methods and cross-checking of the workers' mental model of the task. Traditional HCI design methods are difficult to apply for crowd workers. Although we could have designed the tasks with in-person workers first, that would be ecologically invalid because crowd workers come to the site with more diverse background and knowledge.

Solution. The solution was interesting because it decomposed the cognitive skill into component skills. It is similar manner to how mathematics can be decomposed into component skills. This might be characterized as a skill ladder. However, this was done within a short period of time, as is necessary for crowd tasks. Another interesting facet of our solution was that we acknowledge that some workers would not understand the task, and accepted that we would need to pay them anyway, but identify who they are so that we could focus on the data received from crowd workers who did understand the task. Our tutorial used a combination of cognitive subskills each tested in isolation, and then combined to convey the broader skill that was required.

*Both authors contributed equally to this research.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8095-9/21/05.

<https://doi.org/10.1145/3411763.3443447>

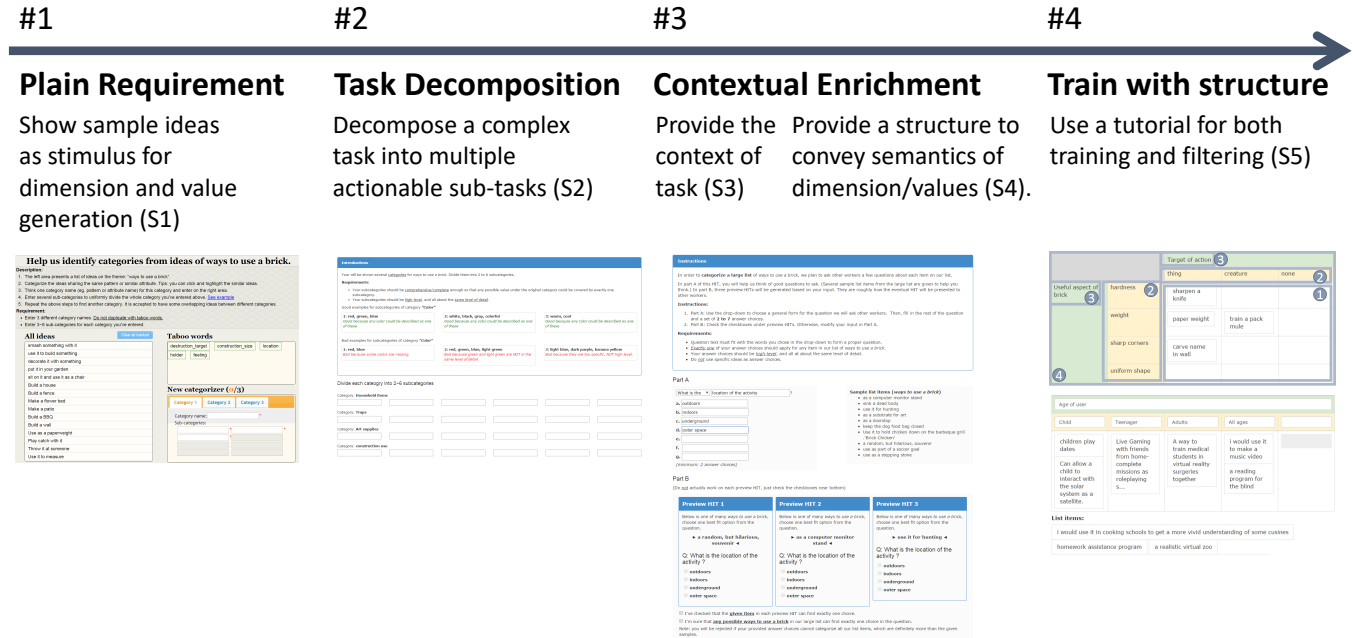


Figure 1: We went through four iterations of the design. In each iteration, we used one or two communication strategies to guide our optimization of the task design. This figure shows the progression of the design that finally reaches to a version (i.e., iteration #4) which effectively communicates the task to workers.

2 BACKGROUND & DEFINITION

In BlueSky [5], the Dimension/Values were generated to help chart the idea space within which workers can be guided to enumerate ideas for a list topic. The D/V was the key to determine if two ideas are distinct or equivalent. Any item in an idea space can be mapped to a set of values for each of the dimensions. Two ideas that map to the same set of values are treated as equivalent.

A *dimension* consists of a label and a set of *values* that can be used to partition the ideas in a list. Therefore, we also call an idea as a *list item*. Take the list topic “ways to use a brick” as an example, some dimensions are inherently discrete and finite (e.g., a dimension “is destructive” with two values {“yes”, “no”}). The others will be clustered into bins or representative values (e.g., a dimension “location” with four values {“indoor”, “outdoor”, “underground”, “outerspace”}).

A *valid* dimension should be able to categorize most, if not all, ideas for the given topic. It also implies that it should be able to categorize any future ideas. On the other hand, a *valid* set of values for a dimension should meet the following criteria as much as possible: 1) *orthogonal* which means no overlap between each other, 2) *evenly-spaced* which means nearly the same distances between each other, and 3) *complete* which means they cover all representative aspects.

3 DESIGN & FINDINGS

Our design went through four iterations before it reached to a point where workers were able to generate *Dimension/Values* that could comprehensively categorize any ideas of a list topic. The timeline of

the iterations is shown in Figure 1. Within each iteration below, we summarize the communication strategies that were used to guide the task design as well as the preliminary findings after testing with workers.

3.1 Iteration #1: Plain Requirement

3.1.1 Design. We began our instruction design by directly asking workers to think of the *Dimension/Values* that could be used to categorize the given list of ideas. Workers were given a list of ideas for a topic (e.g., “ways to use a brick”), the textual description of procedural information that need to perform to complete the task, as well as the requirements of the submission. This represents the communication strategy below.

Strategy: Show sample ideas as stimulus for dimension and value generation (S1). Showing sample ideas allows workers to focus on the generation of dimensions and values, without the need to think of ideas by themselves. Also, the sample ideas were often more diverse than those a worker could think of during a short period of time. Thus the diverse ideas were assumed to be more likely to inspire workers to generate more diverse dimensions and values. Cascade [2] also provided list items and asked workers to generate categories in order to form a taxonomy. However, one difference is that Cascade asked workers to generate one category for each list item, while we did not enforce a one-to-one mapping. Furthermore, one unique challenge is that workers had to provide dimensions and values that can be used to categorize ideas that are not in the list of sample ideas. This is an unusual task that requires relatively complex cognitive skills.

Note that we used the term "Category/Subcategory" rather than "Dimension/Values" in the task description. This was because we assumed that workers may feel more familiar with the former terms, given that categorization is a common task on crowdsourcing platform. However, since "Category/Subcategory" are overloaded terms and sometimes have conflicting meanings depending on the context, we also tried different terms in the later iterations, such as "Question/Choices", "Metric/Values" and "Dimension/Values".

3.1.2 Preliminary findings. We tested this set-up with 16 workers on Amazon Mechanical Turk (AMT), and received 43 dimensions and 156 values. We found several issues from the collected data. First, most dimensions were either invalid (28 out of 43) or irrelevant (11 out of 43) to the given topic (Table 1). Take dimensions of the topic "ways to use a brick" as an example, workers submitted dimensions such as "dangerous" (invalid) or "shape" (irrelevant), which cannot be used to categorize the ideas. By saying a dimension *invalid*, it means that it violates the criteria of a *valid* dimension defined in Section 2. Specifically, the dimension "dangerous" is considered as invalid because it can only be used to categorize a partial set of ideas in "ways to use a brick". The results were first coded by a researcher and then cross-checked by the other researchers.

Second, some dimensions are overly broad (e.g., "use", "usage"). Although these dimensions are reasonable, the granularity level of the corresponding value could either be very broad, or extremely narrow that only cover one idea in the idea space. Besides the issues we found in dimensions, we also observed several problems from the values belonging to a dimension. For example, some values were wrong and did not belong to its dimension. Furthermore, even the values were correct, but they were not complete. As a result, only 7.0% of the D/V was valid. The results indicated that most workers did not thoroughly comprehend the task and submitted dimension and values that were unusable.

3.2 Iteration #2: Task Decomposition

3.2.1 Design. Our first attempt to improve workers' understanding of task was to reduce task complexity. In this iteration, we decomposed the original task into two sub-tasks. Specifically, instead of asking workers to provide dimensions and values at the same time, we simplified the task and asked workers to either provide the dimension for a given list topic or provide values for a given dimension (Figure 2). In other words, workers only needed to focus on one job at one time. We adopted this strategy because we suspected that the original task might be too difficult for workers, given the failure in previous iteration.

Strategy: Decompose a complex task into multiple actionable sub-tasks (S2). Divide-and-conquer is a common algorithm that has been applied to divide a complex crowdsourcing task into several actionable units that could be performed by crowd workers [1, 7]. We did not consider this strategy at first because we thought the complexity of the original task is appropriate for crowd workers. However, after several unsuccessful attempts to optimize the instruction requirement in iteration #1, we suspect that the task might be too difficult to communicate with workers.

3.2.2 Preliminary findings. We tested this set-up with 10 workers on Amazon Mechanical Turk (AMT), and received 35 dimensions and 118 values. As Table 1 shows, we see a significant increase in valid dimensions (9.3% → 42.9%), and decrease in irrelevant dimension (25.6% → 2.9%). However, the "Valid D/V" is merely 28.6%, meaning that a large portion of the values were not aligned with the corresponding dimension, and vice versa. We found that it is very difficult for one worker to generate a dimension that another worker can build on to generate a set of values. This is because the second worker does not have the same understanding of context as the first worker. The second worker is often confused about what values to provide, given that the dimension quality is low. For example, if no values are provided, it is hard to understand the meaning of the dimension created by the first worker, such as "Aspects of Life", and thus hard to provide meaningful values.

Despite this possible confusion from workers, we did receive a better quality of D/V in this iteration, which shows the success of applying decomposition strategy (S2). On the other hand, the disconnection issue between the two sub-tasks (i.e., dimension generation and values generation) indicates that showing more contextual information of the delegated task to workers might help improve workers' understanding of the task, and thus help them to provide results that meet requesters' implicit requirements.

3.3 Iteration #3: Contextual Information Enrichment

3.3.1 Design. Based on the findings from the previous iteration, a new prototype was designed and implemented to improve the quality of dimensions and values. As shown in Figure 3, workers were given a list of ideas as input and requested to think of a good question that can be used to ask **other workers** about each item in the list. Besides, workers were given three HITs that roughly shows how the eventual HIT will be presented to other workers. In other words, workers can *preview* how other workers would be asked to choose an answer for an idea with respect to the questions generated in this step. If they feel their question/choices do not fit with ideas, they are suggested to modify their question/choices. Two potential strategies were considered to guide the design of the new prototype.

Strategy: Provide the context of task (S3). This strategy is mainly inspired by our observation in previous iteration. When workers were finishing part of a complex task, they did not know how their contribution would be used by other workers. By giving the chance of previewing the context, they would be more willing to provide meaningful answers. We implemented three variations for S3, including 1) workers first provide a question and then preview the next step without doing it (as shown in the part B of Figure 3), 2) workers first provide a question and then actually do three previewed HITs of next step, and 3) workers first answer three previewed HITs that have been generated by others and then provide their own question/choices.

Strategy: Provide a structure to convey semantics of dimension/values (S4). We asked the workers to think of a question that is sensible for all the list items. We also provided a set of templates for the question, such as "With whom ___?", "What is the ___?" and "Is the [list topic] ___?" Workers needed to choose one of the template

Iteration	Dimension				
	Valid (%)	Invalid (%)	Irrelevant (%)	Valid D/V (%)	Total number
#1	9.3	65.1	25.6	7.0	43
#2	42.9	54.3	2.9	28.6	35
#3	52.4	39.3	8.3	44.0	84
#4	79.6	16.3	4.1	79.6	49

Table 1: Summary of statistics of Dimension from each Iteration. Note that a “Valid D/V” means the Dimension/Values pair that could be legitimately used to categorize an idea space. A dimension does not count as a valid “Valid D/V” if its corresponding values are invalid. The results were first coded by a researcher and then cross-checked by the other researcher.

Instructions

You will be shown several categories for ways to use a brick. Divide them into 2 to 6 subcategories.

Requirements:

- Your subcategories should be comprehensive/complete enough so that any possible value under the original category could be covered by exactly one subcategory.
- Your subcategories should be high-level, and all about the same level of detail

Examples for subcategories of category “Color”

1: red, green, blue, yellow, black, white <i>Good, because any color could be described as one of these.</i>	2: warm, cool <i>Good, because any color could be described as one of these.</i>
1: red, green, blue, light green <i>Bad, because green and light green are not in the same level of detail.</i>	2: red, yellow <i>Bad, because some color at the same level of detail are missing.</i>

Divide each category into 2-6 subcategories

Category: **Location**

Category: **Art supplies**

Category: **Aspects of Life**

Figure 2: Interface of Iteration #2: Task Decomposition. Workers were asked to generate Values for a corresponding Dimension. Note that we use category/sub-category, not Dimension/Values, in the instructions to make workers more familiar with the term.

and fill the blank. We expected a worker can enter some questions like: "With whom can you use it?", "What is the location of the brick usage?" and "Is the way of using a brick expensive?" Furthermore, for each question, workers needed to enter two or more answer choices. For example, the question "What is the location of the brick usage?" may receive two answer choices: "indoor" and "outdoor".

The formed question is essentially a dimension, while the multiple choices of the question are essentially the values of the dimension. By representing the D/V relationship as a multiple-choice question, we aimed to leverage the intrinsic requirements for a good multiple-choice question: 1) each question should be sensible to the list items; 2) each answer choice should be relevant to the question; and 3) answer choices should be orthogonal (no overlapping), evenly-spaced, and as complete as possible. These requirements are well aligned with the criteria for a valid dimension and a valid set of values (as mentioned in the previous section).

3.3.2 Preliminary findings. We tested this set-up with 36 workers on Amazon Mechanical Turk (AMT), and received 84 dimensions

and 273 values. Some good results were obtained, such as "What is the *time commitment*?" with four choices {"5-20 min", "21-45 min", "46-90 min", "95+ min"} and "What is the *physical effort*?" with three choices {"Little", "Moderate", "Heavy"}. Especially, when workers chose the template "Is the [*list topic*] ___?", they can often provide good *adjective phrases*, such as "Is the way of using a brick *expensive*?" with two choices {"Yes", "No"} and "Is the way of using a brick *fast*?" with two choices {"Yes", "No"}. The high accuracy of questions with adjective phases was largely because the answer choices were boolean values. Although the questions with adjective phrases had high accuracy, we did not encourage workers to solely generate questions in this type, because it is less informative than other types of questions and will cause too many dimensions. For example, a question "What is the *cost*?" with three values {"Low", "Medium", "High"} is equally informative as three questions: "Is the way of using a brick *cheap*?", "Is ... *medium-cost*?", and "Is ... *expensive*?".

As shown in Table 1, we found a 10% increase in valid dimension and 15.4% increase in "Valid D/V", compared to iteration #2.

Instructions

In order to categorize a large list of *ways to use a brick*, we plan to ask other workers a few questions about each item on our list. In part A of this Hit, you will help us think of good questions to ask. In part B, three preview HITs will be generated based on your input. They are roughly how the eventual HIT will be presented to other workers.

Instructions:

- Part A: Use the drop-down to choose a general form for the question we will use to ask other workers. Then, fill in the rest of the question and a set of **2 to 7** answer choices.
- Part B: Check the checkboxes under preview Hits. Otherwise, modify your input in Part A.

Requirements:

- Question text must fit with the words you chose in the drop-down to form a proper question
- Exactly one of your answer choices should apply for any item in our list of ways to use a brick.
- Your answer choices should be *high-level*, and all at about the same level of detail.
- Do not use specific ideas as answer choices.

Part A

What is the location of the activity ?

a. outdoors

b. indoors

c. underground

d.

Part B Preview HIT

Below is one of many ways to use a brick, choose one best fit option from the question.

▶ Use it for hunting ◀

Q: What is the location of the activity?

☐ outdoors

☐ indoors

☐ underground

Figure 3: Interface of Iteration #3: Contextual Enrichment. We provide more contextual information of the task by letting workers know how their submission (Part A) would be used by other workers (Part B).

However, the dimensions (i.e., questions) and values (i.e., answer choices) still suffered from the issues of prior iterations. For example, some questions were irrelevant to the list topic (e.g., "What is the Capital of Canada?" for the topic "ways to use a brick") while some questions were too broad (e.g., "Is ... good?"). Additionally, some questions like "What is the most creative way?" were generated in which workers listed out several list items as answer choices. In this case, the answer choices of the question could not be used to categorize any items in the list.

We also found some issues different from prior iterations. Some questions like "What is the *best season for usage*?" with four choices {"Spring", "Summer", "Autumn", "Winter"} seemed to be a valid D/V. However, those values were not complete. Many list items of ways to use a brick do not have a clear "best" season for use. They can be used in full year. Similarly, a question was "What is the best time of day of usage?"

All the issues above were assumed to be resolvable by S3 but were not completely resolved. Showing previews of next step (Figure 3) did not help some workers to realize that their question/choices were irrelevant, too broad, or incomplete. The three variations of S3 did not have significant differences. We started to acknowledge that only some workers could pay enough attention to the requirements and fully understand the task, while the rest could understand partially or none.

3.4 Iteration #4: Training with Structure

In this iteration, we synthesized the lessons learned from iteration #1-#3 and designed the final version. We provided a structured mini-tutorial to guide workers through the confusion of generating Dimension/Values.

As Figure 4 shows, we decomposed the original task into sub-tasks (S2) and guided the workers to understand the relationship of D/V by presenting it as a table (S4). The tutorial contained four steps. The first step (Figure 4A) asked workers to categorize a small list of items by the given dimensions and values (S3). The second step (Figure 4B) asked them to enter values for given dimensions and already categorized list items. Next, the third step (Figure 4C) asked them to enter dimensions for given values and already categorized list items. Steps 1-3 have gradually demonstrated all the necessary subskills—categorization, generating dimensions, and generating values—for the actual D/V generation task. In step 4, workers were given a list of uncategorized items and asked to think of dimensions, values and finally categorize the items (Figure 4D). For the first three steps, we knew the ground truth so that if a worker completed one step, we provided confirmation for correctness or reasons for mistakes. For step 4, the results will be manually checked by a qualified worker who has passed the tutorial. To make the tutorial more generalizable and skill more transferable, each step used the same list topic but different dimensions, values and list items.

Strategy: Use a tutorial for both training and filtering (S5). In previous iteration, we found that some top-workers managed to contribute most of the meaningful D/V, while some workers could not understand the task and submitted subpar D/V. Therefore, the mini-tutorial was used to not only teach the skills but also identify workers who can generate legitimate D/V.

3.4.1 Preliminary findings. We tested this set-up with 30 workers on AMT and 13 workers passed the tutorial who then generated 49 dimensions and 200 values. We found that the rate of valid dimension greatly increases from 52.4% to 79.6%. More importantly, the "Valid D/V" (79.6%) is same as the valid dimension rate, meaning that all the valid dimensions have corresponding values that can

A

		Useful aspect of brick		
		Hardness	Weight	Sharp corners
Target of action	Thing	<i>Categorize a list of "ways to use a brick" by given dimensions and values.</i>		
	Create ure			
	None			

B

		Useful aspect of brick		
		<i>Enter Values</i>		
Target of action	Enter Values	Sharpen a knife	Paper weight	Crave name in wall
			Train a pack mule	

C

		<i>Enter Dimensions</i>		
		Hardness	Weight	Sharp corners
Enter Dimensions	Thing	Sharpen a knife	Paper weight	Crave name in wall
	Create ure		Train a pack mule	
	None			

D

		<i>Enter Dimensions</i>		
		<i>Enter Values</i>		
Enter Dimensions	Enter Values	Paper weight	Sharpen a knife	Crave name in wall
		Train a pack mule		

Figure 4: Interface of mini-tutorial in Iteration #4. First, workers were given dimensions/values and asked to categorize the given list of ideas. Next, they were asked to enter values to each dimension that can validly categorize the ideas in the table. Third, they were asked to think of two dimensions given the values and a list of categorized ideas. Finally, they were given a list of ideas, and asked to find the dimensions and values.

be used for categorizing the idea space. The main reason was that the necessary skills have been partitioned into the first three steps so that workers can build up skills gradually and also the filtering was very effective. Note that all workers were trained with the same list topic (i.e., "ways to use a brick"), but they were asked to generate D/V for different topics (e.g., "debate questions" and "interview questions"). The high rate of valid D/V indicated that workers were able to transfer their skills from one topic to others.

4 DISCUSSIONS

In this section, we discuss the strategies used to optimize the results, as well as some strategies that we considered but did not implement.

Why not filter out invalid dimensions with extra tasks, like Cascade [2]? Our primary goal was to make as many workers to understand the task as possible and generate good dimensions without extra filtering steps. In other words, we still have the expectation that optimizing the instruction design can eventually make more workers understand the task and provide better results. Also, with fewer steps, it will be more cost-effective. Otherwise, the fact that only a small portion of received dimensions are valid would make the dimension-filtering very difficult and expensive. Also, note that S5 involves filtering, which is different because it was filtering workers rather than dimensions.

Why not use strategy: iterative improvement? Iterative improvement is a general crowdsourcing pattern that uses multiple workers to build on and improve each other's work, which has shown success in image description and decipher blurry text [8]. However, Cascade [2] showed negative outcome in taxonomy creation because workers had trouble selecting which part to contribute. Our task involves even more options than Cascade (e.g., add category, refine category, add value, edit value, delete value,

etc.). We suspected the same issue would happen in our task so that we did not leverage it.

Decompose complex task into subskills but show the big picture to workers. Although the divide-and-conquer algorithm is widely applied to partition the complex task into small tasks, workers are often hidden from what the overall task is about. Sometimes requesters might be afraid of complicating the task and choose not to reveal too much information about the task. However, our results suggest that it may be beneficial to provide some contextual information of the task to workers. This could potentially help them understand the task better and comprehend the implicit requirements that are hard to specify.

Use a structure to convey the requirements implicitly. Both the question-based (in iteration #3) or table-based (in iteration #4) were used to convey the requirements that 1) a dimension has several values, 2) values should be evenly distributed without overlapping, and 3) values should be as comprehensive as possible. Such structures can be used to convey semantics that would otherwise be hard to convey in words.

Generalizability of the tasks. In this case study, we mainly focus on a cognitive task that requires crowd workers to generate D/V for a list topic. Nonetheless, many crowdsourcing tasks in industry or academia share a similar set of challenges: 1) involving multiple steps where the output of one step is used as input for the next; 2) no dialogue between the requester and the workers in which any misunderstanding or confusion cannot easily be resolved or prevented; and 3) tasks (including understanding their instructions) should be completed in a limited time. Examples of those tasks include: exploring different designs of graphic Web advertisements [3], ideating alternative website UIs, and composing different short stories. Those tasks can benefit from our proposed strategies to enhance the communication and receive better responses.

5 CONCLUSION

This case study demonstrates the evolution of a task design engaged crowd workers to generate Dimension/Values that as part of a crowd-powered system for exploring an idea space. The greatest challenge was to help workers understand what we needed them to do; the task was unfamiliar and required relatively complex cognitive skills.

This paper presented the strategies and preliminary findings of four iterations of our task design, over which the quality of Dimension/Values improved gradually. The lessons in the design process shed light on the design of future crowdsourcing systems that require relatively complex cognitive skills.

ACKNOWLEDGMENTS

This work was partially supported by NSF FW-HTF 1839971 as well as Purdue University Frederick N. Andrews Fellowship.

REFERENCES

- [1] Michael S. Bernstein, Greg Little, Robert C. Miller, Björn Hartmann, Mark S. Ackerman, David R. Karger, David Crowell, and Katrina Panovich. 2015. Soylent: A Word Processor with a Crowd Inside. *Commun. ACM* 58, 8 (July 2015), 85–94. <https://doi.org/10.1145/2791285>
- [2] Lydia B. Chilton, Greg Little, Darren Edge, Daniel S. Weld, and James A. Landay. 2013. Cascade: Crowdsourcing Taxonomy Creation. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '13)*. Association for Computing Machinery, Paris, France, 1999–2008. <https://doi.org/10.1145/2470654.2466265>
- [3] Steven P. Dow, Alana Glassco, Jonathan Kass, Melissa Schwarz, Daniel L. Schwartz, and Scott R. Klemmer. 2011. Parallel Prototyping Leads to Better Design Results, More Divergence, and Increased Self-Efficacy. *ACM Trans. Comput.-Hum. Interact.* 17, 4, Article 18 (Dec. 2011), 24 pages. <https://doi.org/10.1145/1879831.1879836>
- [4] Ujwal Gadiraju, Besnik Fetahu, Ricardo Kawase, Patrick Siehdnel, and Stefan Dietze. 2017. Using Worker Self-Assessments for Competence-Based Pre-Selection in Crowdsourcing Microtasks. *ACM Trans. Comput.-Hum. Interact.* 24, 4, Article 30 (Aug. 2017), 26 pages. <https://doi.org/10.1145/3119930>
- [5] Gaoping Huang and Alexander J. Quinn. 2017. BlueSky: Crowd-Powered Uniform Sampling of Idea Spaces. In *Proceedings of the 2017 ACM SIGCHI Conference on Creativity and Cognition (C&C '17)*. ACM, Singapore, Singapore, 119–130. <https://doi.org/10.1145/3059454.3059481>
- [6] Jordan S. Huffaker, Jonathan K. Kummerfeld, Walter S. Lasecki, and Mark S. Ackerman. 2020. Crowdsourced Detection of Emotionally Manipulative Language. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, Honolulu, HI, USA, 1–14. <https://doi.org/10.1145/3313831.3376375>
- [7] Anand Kulkarni, Matthew Can, and Björn Hartmann. 2012. Collaboratively Crowdsourcing Workflows with Turkomatic. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work (Seattle, Washington, USA) (CSCW '12)*. Association for Computing Machinery, New York, NY, USA, 1003–1012. <https://doi.org/10.1145/2145204.2145354>
- [8] Greg Little, Lydia B. Chilton, Max Goldman, and Robert C. Miller. 2010. TurkKit: Human Computation Algorithms on Mechanical Turk. In *Proceedings of the 23rd Annual ACM Symposium on User Interface Software and Technology (UIST '10)*. Association for Computing Machinery, New York, NY, USA, 57–66. <https://doi.org/10.1145/1866029.1866040>
- [9] Winter Mason and Duncan J. Watts. 2009. Financial Incentives and the "Performance of Crowds". In *Proceedings of the ACM SIGKDD Workshop on Human Computation (HCOMP '09)*. Association for Computing Machinery, Paris, France, 77–85. <https://doi.org/10.1145/1600150.1600175>